

2º Semestre - 2025/2026

Unidade Curricular:

Big Data

London Bikes

Análise de Padrões de Mobilidade Urbana

Trabalho realizado por Grupo 2:

Gonçalo Henriques, nº 123422;

Rodrigo Sousa, nº 123390.

Docente: Paulo Vieira

Lisboa, Maio de 2026

Índice

1. INTRODUÇÃO	3
2. METODOLOGIA	4
2.1 <i>Datasets</i> Utilizados	4
2.2 Preparação dos Dados	4
2.3 Técnica de Modelação	5
3. ANÁLISE DE RESULTADOS	5
3.1 Qualidade do Modelo	5
3.2 Distribuição da Capacidade de Docas	5
3.3 Matriz de Correlação	6
3.4 <i>Scatter Plot</i> dos <i>Clusters</i>	7
3.5 Padrões de Mobilidade Identificados	7
4. IMPLEMENTAÇÃO TÉCNICA	8
4.1 Arquitetura do Sistema	8
4.2 Fluxo de Dados	8
4.3 <i>Dashboard</i> Interativo	9
4.4 Desafios e Soluções	9
5. CONCLUSÕES E TRABALHO FUTURO	9
6. APÊNDICE TÉCNICO	10
6.1 <i>MongoDB</i>	10
6.2 <i>PySpark</i>	10
6.3 Como se Complementam	11
6.4 Principais Funções e Comandos Utilizados	11
FONTE DE DADOS	12

1. INTRODUÇÃO

Este relatório descreve o desenvolvimento de uma solução de análise de mobilidade urbana com base no *dataset* [London Cycles](#). O projeto usa *Apache Spark* para processamento distribuído, *MongoDB* para persistência dos dados tratados e modelados, *Docker* para encapsular o ambiente de execução e *Streamlit* para exploração interativa dos resultados.

Os sistemas de bicicletas partilhadas geram grandes volumes de dados temporais e geográficos, o que os torna adequados para um projeto académico de Big Data. O *dataset* London Cycles contém observações históricas de disponibilidade de bicicletas e docas por estação, permitindo estudar padrões de utilização e comportamento da rede ao longo do tempo.

O problema abordado consiste em identificar o estado operacional das estações da rede de bicicletas partilhadas de Londres em cada instante observado. Pretende-se distinguir estações em equilíbrio, estações com escassez de bicicletas e estações com excesso de bicicletas relativamente ao número de docas livres. Esta formulação conduz naturalmente a uma abordagem não supervisionada: segmentar as observações em grupos com comportamento semelhante, recorrendo a *clustering* sobre as variáveis *bikes*, *empty_docks* e *docks*.

Os objetivos para a elaboração do projeto passaram por:

- Ingestão e normalização dos dados de bicicletas e de estações a partir de múltiplos ficheiros CSV;
- Construção de *collections* limpas e reutilizáveis em MongoDB;
- Aplicação de K-Means às observações de verão para segmentar estados de utilização;
- Integração de *bikes_clean_summer* com *stations_clean* para obter o nome comum de cada estação;
- Disponibilização de um *dashboard* com navegação temporal ao nível do minuto.

2. METODOLOGIA

2.1 Datasets Utilizados

O *dataset* principal provém do *Kaggle* (London Cycles) e inclui dois tipos de ficheiros: observações de disponibilidade de bicicletas por estação (um CSV por dia, com granularidade aproximada de 15 em 15 minutos) e *snapshots* administrativos da rede de estações (*common_name*, coordenadas, capacidade). Os dados de bicicletas são identificados por *query_time* e incluem os campos *place_id*, *bikes*, *empty_docks* e *docks*. Após combinar todos os CSVs diários em Parquet, verificamos que o volume total ultrapassa os 22 milhões de observações distribuídas pelos quatro períodos sazonais.

2.2 Preparação dos Dados

A limpeza foi realizada com *PySpark* e produziu as seguintes *collections* em MongoDB:

Collection	Descrição	Documentos
<i>stations_raw</i>	Snapshots brutos de estações	385 731
<i>stations_clean</i>	Dimensão limpa de estações	812
<i>bikes_clean_summer</i>	Observações limpas de verão	3 048 965
<i>bikes_clean_autumn</i>	Observações limpas de outono	5 909 122
<i>bikes_clean_spring</i>	Observações limpas de primavera	6 651 206
<i>bikes_clean_winter</i>	Observações limpas de inverno	6 471 650
<i>bikes_modelled_summer</i>	Observações modeladas de verão	3 048 965

As operações de limpeza aplicadas foram:

- Remoção de registos com *docks* = 0;
- Remoção de observações em que *bikes* + *empty_docks* ≠ *docks*;
- Correção de coordenadas *lat* = 0 e *lon* = 0;

- Uniformização de coordenadas para *BikePoints* com múltiplas localizações históricas;
- Remoção de duplicados de *stations_raw* por *place_id*, mantendo o estado mais recente;
- Converter tipos textuais para booleanos e *timestamps Unix* em *ISODate*;
- Preservar a granularidade temporal real: *query_time*, *Date*, *Hour* e *Minute*.

2.3 Técnica de Modelação

A modelação aplica *K-Means* a cada observação individual da *collection bikes_clean_summer*, usando as variáveis *bikes*, *docks* e *empty_docks*. A pipeline *MLlib* inclui três etapas em sequência:

- *VectorAssembler*: combina as três variáveis num vetor de *features*;
- *StandardScaler*: normaliza a escala para que nenhuma variável domine o *clustering*;
- *KMeans*: $k = 3$, *seed* = 42, medida euclidiana.

O valor $k = 3$ foi escolhido com base no domínio do problema: existem naturalmente três estados operacionais possíveis — utilização frequente, moderada e excessiva. A nomeação automática dos *clusters* foi feita pelo perfil médio de cada grupo: o *cluster* com mais *empty_docks* foi classificado como excessivo, o com mais *bikes* como moderado e o restante como frequente.

3. ANÁLISE DE RESULTADOS

3.1 Qualidade do Modelo

O modelo *K-Means* obteve um *Silhouette* aproximado de 0.56, indicador de separação aceitável entre os três *clusters*. A *collection bikes_modelled_summer* preserva *query_time*, *Date*, *Hour*, *Minute*, *place_id*, *common_name*, *lat*, *lon* e as variáveis de estado operacional (*cluster_id* e *nivel_utilizacao*).

3.2 Distribuição da Capacidade de Docas

A distribuição da capacidade mostra que a maioria das estações se concentra numa faixa relativamente estreita de docas totais. Isto sugere uma rede com alguma padronização

estrutural, mesmo admitindo a existência de estações pequenas e outras bastante maiores. A variável *docks* representa uma restrição física real do sistema e não apenas ruído.

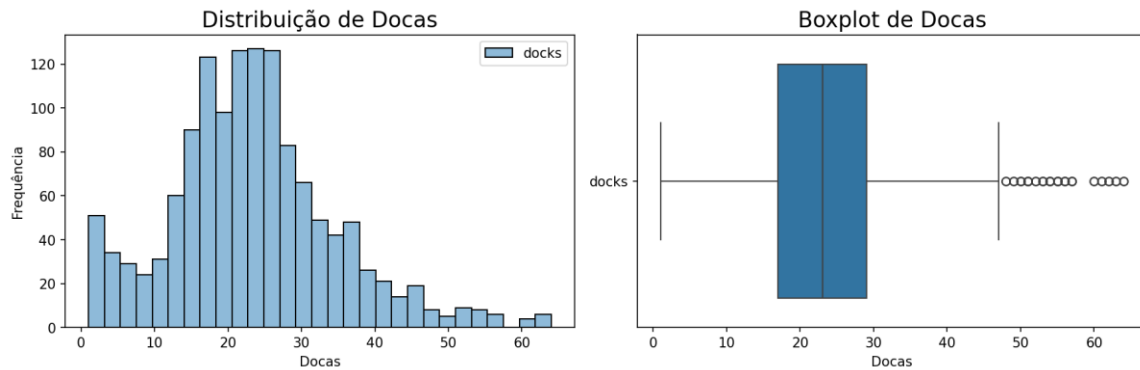


Figura 1 – Distribuição do número de docas por estação.

3.3 Matriz de Correlação

A matriz de correlação confirma a intuição do domínio: *bikes* e *empty_docks* tendem a variar em sentidos opostos, enquanto *docks* funciona como a capacidade total que condiciona ambas. Esta relação justifica a escolha das três variáveis como base do *clustering*, porque juntas descrevem o equilíbrio operacional de cada estação.

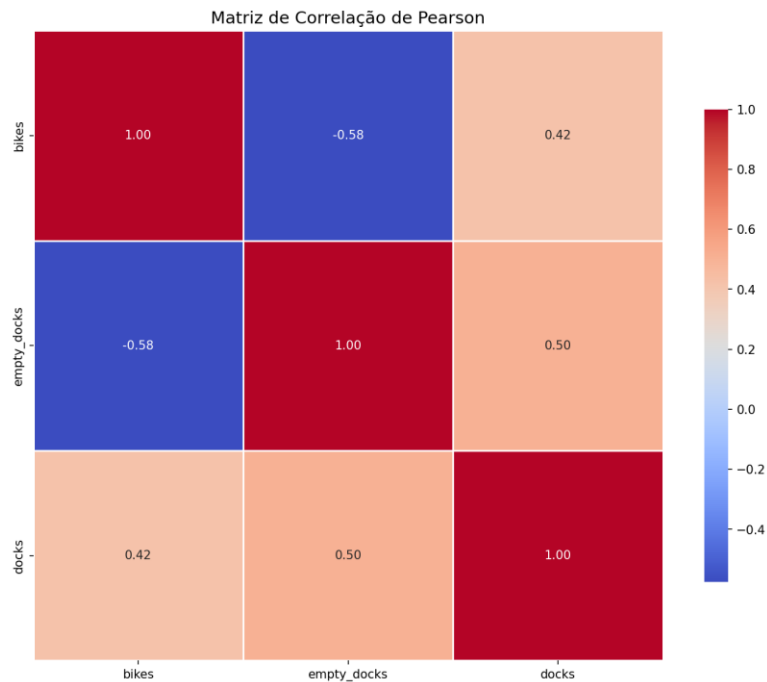


Figura 2 – Correlação entre *bikes*, *empty_docks* e *docks*.

3.4 Scatter Plot dos Clusters

O *scatter plot* mostra uma separação clara entre zonas do plano: docas vazias e poucas bicicletas, muitas bicicletas e poucas docas livres, e uma região intermédia mais equilibrada. Esta distribuição confirma a interpretação operacional dos três *clusters*:

- *Cluster* Frequente: observações próximas do equilíbrio entre bicicletas disponíveis e docas vazias;
- *Cluster* Excessivo: observações com muitas docas vazias e poucas bicicletas, indicando escassez de oferta local;
- *Cluster* Moderado: observações com muitas bicicletas e poucas docas vazias, favorável para recolha mas menos para devolução.

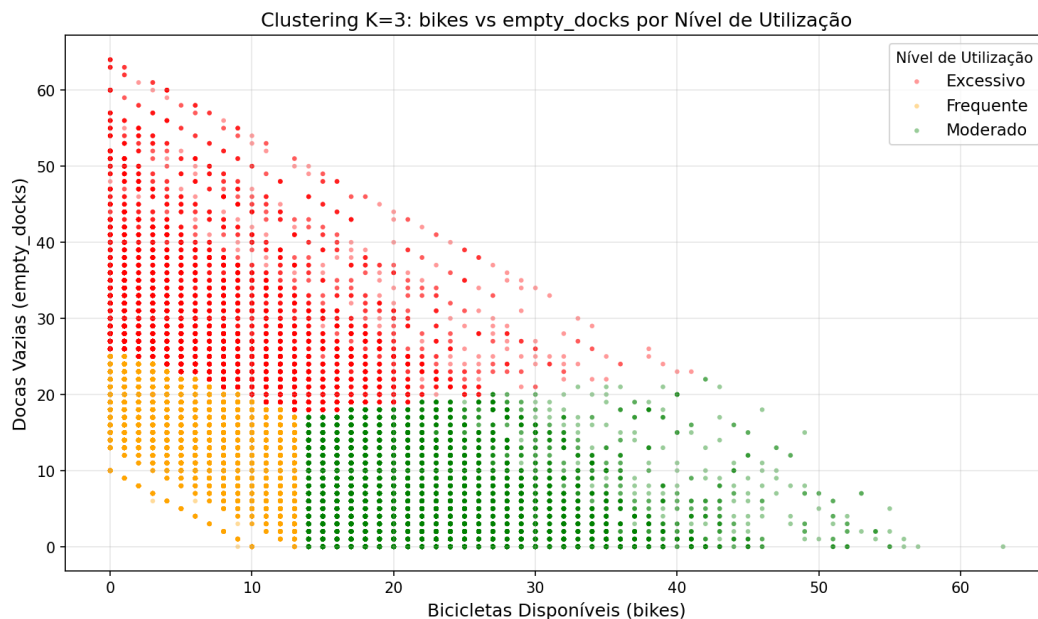


Figura 3 – Dispersão dos clusters em bikes vs empty_docks.

3.5 Padrões de Mobilidade Identificados

As visualizações mostram que os grupos encontrados possuem interpretação operacional e são compatíveis com o comportamento esperado de uma rede de bicicletas partilhadas. O estado Excessivo tende a ocorrer em estações periféricas ou em horários de menor movimento, enquanto o estado Moderado domina nas estações centrais durante as horas de pico. O *dashboard Streamlit* permite explorar esta distribuição temporal e geograficamente.

4. IMPLEMENTAÇÃO TÉCNICA

4.1 Arquitetura do Sistema

A arquitetura pode ser entendida como uma cadeia de processamento e persistência. Os dados brutos são lidos por *Spark*, transformados em *DataFrames*, guardados em *Parquet* para eficiência analítica, persistidos em MongoDB para consulta operacional e consumidos por uma aplicação *Streamlit* com *Folium*.

Camada	Tecnologia	Papel no projeto
Orquestração	<i>Docker Compose</i>	Cria e liga os serviços Jupyter/PySpark e MongoDB numa rede comum, com volumes persistentes.
Processamento	<i>Apache Spark (PySpark)</i>	Ler, limpar, transformar e modelar milhões de observações com DataFrames e MLlib.
Armazenamento analítico	<i>Parquet</i>	Guardar os dados de bicicletas num formato colunar eficiente para leitura repetida em Spark.
Persistência operacional	<i>MongoDB</i>	Guardar collections limpas, dimensão de estações e resultados do clustering para consulta rápida.
Visualização	<i>Streamlit + Folium</i>	Disponibilizar exploração interativa temporal e geográfica sobre a collection modelada.

4.2 Fluxo de Dados

- *Kaggle/CSV* → *Spark*: os ficheiros diários de bicicletas e *snapshots* de estações são lidos no *notebook* de ingestão;
- *Spark* → *Parquet*: os dados de bicicletas são guardados em formato de colunas para leitura repetida e eficiente;
- *Spark* → *MongoDB*: os subconjuntos limpos e os resultados de modelação são persistidos em *collections* reutilizáveis;

- *MongoDB* → *Streamlit*: a aplicação final consulta diretamente *bikes_modelled_summer* para visualização interativa;
- *Docker Compose*: garante rede interna, volumes persistentes e reprodutibilidade do ambiente.

4.3 Dashboard Interativo

O *dashboard* foi implementado em *Streamlit*, com leitura direta da *collection bikes_modelled_summer*. A aplicação suporta filtros por data, hora e minuto, permitindo navegar pelos *snapshots* temporais reais. O uso de *common_name*, herdado da integração com *stations_clean*, melhora significativamente a legibilidade da interface.

- Mapa interativo em *Folium* com estações coloridas por *nivel_utilizacao*;
- *Tooltip* e *popup* com *common_name* e métricas da observação;
- Gráfico de barras com contagem de observações por nível;
- Tabela com *timestamp*, estação, estado e indicadores de disponibilidade.

4.4 Desafios e Soluções

O projeto envolveu vários desafios técnicos típicos de pipelines de dados reais:

- Volume e fragmentação do *dataset*: múltiplos CSVs independentes foram combinados num único *Parquet* antes de qualquer transformação;
- Qualidade irregular dos dados: coordenadas inválidas, registos inconsistentes e múltiplas localizações históricas para algumas estações;
- Gestão de memória do *Spark* em *Docker/WSL2*: ajuste de configurações de *executor memory* e *driver memory*;
- Remoção de duplicados e normalização da dimensão de estações: a *collection stations_raw* contém vários *snapshots* da mesma estação ao longo do tempo;
- Preservação da granularidade temporal até ao *dashboard*: manter *query_time* e *Minute* em todas as etapas da pipeline.

5. CONCLUSÕES E TRABALHO FUTURO

O projeto atingiu os objetivos propostos: foi construído um pipeline completo de *Big Data*, desde a ingestão até à visualização final, com apoio de *Spark*, *MongoDB*, *Docker* e *Streamlit*. A solução produz *datasets* limpos, uma dimensão de estações reutilizável, uma

collection modelada para o verão e um *dashboard* que permite explorar o estado das estações ao longo do tempo.

Do ponto de vista analítico, o *K-Means* mostrou-se adequado para segmentar os estados operacionais das estações com base nas variáveis de disponibilidade. Do ponto de vista de engenharia de dados, a preservação da granularidade temporal ao nível do minuto e a integração com *stations_clean* foram as melhorias mais importantes da versão final.

Como trabalho futuro, faria sentido:

- Estender a modelação às restantes estações do ano e comparar distribuições sazonais dos clusters.
- Explorar modelos preditivos para antecipar situações de utilização excessiva.
- Incorporar dados meteorológicos para estudar correlações entre condições climáticas e padrões de utilização.
- Aumentar o número de estações cobertas pelo dashboard com filtros por zona geográfica.

6. APÊNDICE TÉCNICO

6.1 *MongoDB*

MongoDB é uma base de dados *NoSQL* orientada a documentos com esquema flexível. As principais vantagens são a integração nativa com *Spark* via conector oficial, o suporte a dados geoespaciais e a capacidade de escalar horizontalmente. É tipicamente usado em sistemas de *IoT*, análise de eventos em tempo real e aplicações com dados heterogêneos. Neste projeto serve como camada de persistência para todas as *collections* limpas e para os resultados de modelação.

6.2 *PySpark*

PySpark é a interface *Python* do *Apache Spark*, um *framework* de computação distribuída que processa grandes volumes de dados em memória. As principais vantagens são o processamento em memória (mais rápido que *MapReduce*), a API unificada para *batch*, *SQL* e *machine learning*, e a biblioteca *MLlib* com algoritmos como *K-Means*. É tipicamente usado em pipelines ETL e *machine learning* distribuído. Neste projeto foi

usado para combinar e limpar os CSVs diários, executar o modelo *K-Means* e escrever os resultados em *MongoDB*.

6.3 Como se Complementam

Spark processa e transforma dados em grande escala mas não serve como base de dados operacional. *MongoDB* persiste os resultados e responde rapidamente a consultas da aplicação mas não foi desenhado para análise massiva. A combinação é natural: *Spark* transforma e modela, *MongoDB* armazena e serve. O conector *mongo-spark-connector* concretiza esta ligação, permitindo que o *Spark* leia e escreva *collections* diretamente sem ficheiros intermédios.

6.4 Principais Funções e Comandos Utilizados

Operação	Função / API	Notebook
Leitura de CSV	<code>spark.read.csv(..., inferSchema=True, header=True)</code>	1
Escrita Parquet	<code>df.write.mode('overwrite').parquet(path)</code>	1
Leitura MongoDB	<code>spark.read.format('mongodb').option('uri',...).load()</code>	2, 3, 4
Escrita MongoDB	<code>df.write.format('mongodb').option('collection',...).save()</code>	1, 2, 3, 4
Remoção de nulos	<code>df.dropna(subset=[...])</code>	2
Filtragem	<code>df.filter(condition)</code>	2, 4
Vetor de features	<code>VectorAssembler(inputCols=[...], outputCol='features')</code>	4
Normalização	<code>StandardScaler(inputCol='features', outputCol='scaled')</code>	4
Clustering K-Means	<code>KMeans(k=3, seed=42, featuresCol='scaled')</code>	4
Avaliação	<code>ClusteringEvaluator(metricName='silhouette').evaluate(df)</code>	4
Pipeline MLlib	<code>Pipeline(stages=[assembler, scaler, kmeans]).fit(df)</code>	4
Join	<code>df1.join(df2, on='place_id', how='left')</code>	4
Adicionar coluna	<code>df.withColumn('nivel', when(...).otherwise(...))</code>	4

FONTE DE DADOS

Ioexception (2023). London Cycles. Kaggle. Disponível em:
<https://www.kaggle.com/datasets/ioexception/london-cycles/data> [Acedido em abril de
2026].